Homework 4 Weakest Precondition Loops

For each proof I specify the rule above and the application below it.

Problem 1

Proof

1. Invariant initialization: the invariant must be true before entering the loop
    - $I = m0! = 0 \wedge n0 * m0 <= usize :: MAX \wedge n0 * m0 == res + n * m0$
    - 0: I
2. Variant Lower Bound ($\forall \vec{x} \left[c \wedge I \implies wp([s], d \geq 0) \right]$)
    - 1: $I \wedge 0 < n$
3. Assume the function call holds.
    - 2: $I \wedge res + m0 <= n0 * m0$
    - for all n, res ==> (2)
4. Assignment rule applied $wp([x := e], Q) = Q[e/x]$:
    - 3: $I \wedge res + m0$
    - 4: $I \wedge n - 1$
    - 5: $I == (res + m0) + (n - 1) * m0$
5. Assume (Partial Correctness): $wp([assume\ e], Q) = e \wedge Q$
    - for all n, res ==> (4)
6. Assume the function call holds.
    - 6: $I \wedge n0 * m0 == res + n * m0$

Code

```
pub proof fn lemma_q3(n0: usize, m0: usize, res: usize, n: usize)
{
    // I (invariant)
    // assert requires implies invariant
    assert((m0 != 0 && n0 * m0 <= usize::MAX) ==> (m0 != 0 && n0 * m0 <=
usize::MAX && n0 * m0 == res + n * m0));

    // while loop body preserves I
    assert((0 < n) ==> (m0 != 0 && n0 * m0 <= usize::MAX && n0 * m0 == res + n *
m0 && res + m0 <= n0 * m0));
    assert((0 < n) ==> (n0 * m0 == (res + m0) + (n-1) * m0) && ((res+m0) + m0 <=
n0 * m0));
```

```
    // not looping implies Q
    assert((0 < n) ==> (res == 0));
    // termination
    assert((n > 0));
}
```

Problem 2

Proof

1. Assume the invariant
   - $I = n * (n + 1)/2 == sum$
   - 0: I
2. Conditional rule (if statement):
   - 1: branch_then: $n == 0 ==> 0$
   - 2: branch_else: n != 0 ==> n + sum_to_n_rec_impl((n-1) as usize)
3. Assignment rule: (n-1)
   - 3: n - 1 != 0 ==> n -1 + sum_to_n_rec_impl((n-1)-1 as usize)

Code

```
mod problem_2 {
    use vstd::prelude::*;

    pub exec fn sum_to_n_rec_impl(n: usize) -> (sum: usize)
        requires
            n * (n + 1) / 2 <= usize::MAX,
        ensures
            n * (n + 1) / 2 == sum,
        decreases n,
    {
        if n == 0 {
            0
        } else {
            n + sum_to_n_rec_impl((n-1) as usize)
        }
    }

    pub proof fn lemma_sum_to_n_rec_impl(n: usize, sum: usize) {
        // Assert the invariant
        assert(n * (n + 1) / 2 == sum);

        // not looping implies Q
        if n == 0 {
            assert(sum == 0);
        } else {
            // Prove while loop preserves the invariant
            let sum_prev = sum_to_n_rec_impl((n-1) as usize);
            assert(sum == n + sum_prev);
            lemma_sum_to_n_rec_impl((n-1) as usize, sum_prev);
```

```
            assert(sum == n * (n + 1) / 2);
        }
    }

}
```

Problem 3

Proof

1. Assume the invariant
   - $I = i <= n \wedge acc == i * (i+1)/2 \wedge n * (n+1)/2 <= usize :: MAX$
   - 0: I
2. loop body preserves I
   - 1: $i <= n \wedge (acc + i) == i * (i+1)/2 \wedge n * (n+1)/2 <= usize :: MAX$
   - 2: $(i+1) <= n \wedge (acc + (i+1)) == (i+1) * ((i+1)+1)/2 \wedge n * (n+1)/2 <= usize :: MAX$
   - for all n, acc ==> (2)
3. not going through loop, not looping implies Q
   - 3: $!(i < n) ==> i <= n \wedge acc == i * (i+1)/2 \wedge n * (n+1)/2 <= usize :: MAX$
4. termination
   - for all i ==> (3)

Code

```
mod problem_3 {
    use vstd::prelude::*;

    pub exec fn sum_to_n_iter_impl(n: usize) -> (sum: usize)
        requires
            n * (n + 1) / 2 <= usize::MAX,
        ensures
            n * (n + 1) / 2 == sum,
        decreases n,
    {
        let mut i: usize = 0;
        let mut acc: usize = 0;

        while i < n
            invariant
                i <= n,
                acc == i * (i + 1) / 2,
                n * (n + 1) / 2 <= usize::MAX,
            decreases
                n - i,
        {
            i = i + 1;
            acc = acc + i;
        }
```

```
        acc
    }

    pub proof fn lemma_sum_to_n_iter_impl(n: usize, i: usize, acc: usize) {
        // Assume the invariant holds
        assert(i <= n && acc == i * (i + 1) / 2 && n * (n + 1) / 2 <= usize::MAX);
        // loop body preserves the invariant
        assert(i < n ==> ((i+1) <= n && (acc + (i+1)) == (i+1) * ((i+1) + 1) / 2
&& n * (n + 1) / 2 <= usize::MAX));
        // not going through loop, not looping implies Q
        assert(!(i < n) ==> (i <= n && acc == i * (i + 1) / 2 && n * (n + 1) / 2
<= usize::MAX));
        // termination
        assert(i < n ==> (n - i > 0));
    }
}
```